Практическая 2 Ветвящийся вычислительный процесс

Разветвляющимся называется алгоритм, в котором действие выполняется по одной из возможных ветвей решения задачи, в зависимости от выполнения условий. В отличие от линейных алгоритмов, в которых команды выполняются последовательно одна за другой, в разветвляющиеся алгоритмы входит условие, в зависимости от выполнения или невыполнения которого выполняется та или иная последовательность действий.

Разветвляющиеся алгоритмы можно реализовать с помощью двух операторов: условного оператора и оператора выбора.

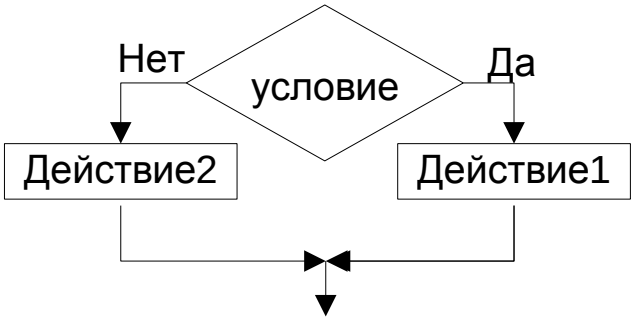
Условный оператор

Теория

Назначение условного операторы – обеспечить ветвление алгоритма в зависимости от выполнения некоторого условия.

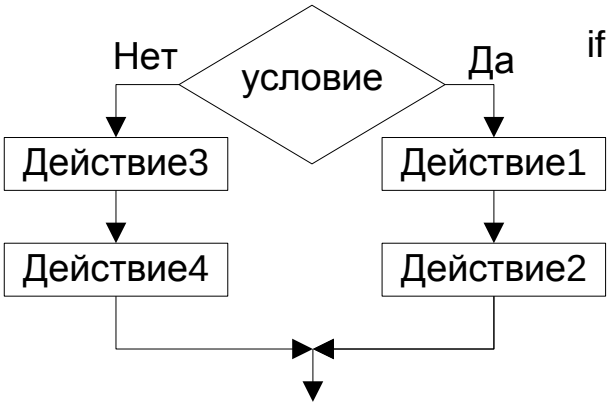
Блок проверки условия (блок принятия решения): изображается ромбом с двумя выходами «Да» и «Нет».

В языке C++ условный оператор имеет следующий вид: Если условие, записанное после служебного слова .if, верно, то выполняется действие 1, иначе – действие 2.



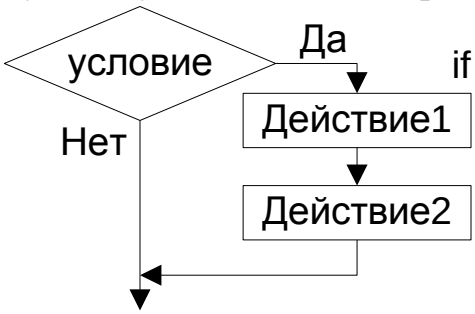

```
if (условие)
        Действие1;
    else
        Действие2;
```

Если на ветках «Да» или «Нет» надо выполнять несколько действий, эти операторы надо заключать в операторные скобки { и }.




```
if (условие) {
    Действие1;
    Действие2;
 }
else
 {
    Действие3;
    Действие4;
 };
```

Существует не полный вариант оператора if, когда ветка «Нет» пустая.




```
if (условие) {
    Действие1;
    Действие2;
};
```

Условие представляет собой логическое выражение. **Логическое выражение** — это любое выражение, которое может быть либо истинным, либо ложным. Логическое выражение состоит из одного или нескольких отношений. Операция отношения выполняет сравнение двух операндов. Результат операции отношения значение булевского типа true или false.

Основные операции отношения:

- == равно;
- < > не равно;
- >больше;
- < меньше; >=
- >=больше или равно; <=
- <=меньше или равно.

Если условие имеет более сложный вид, т. е. содержит несколько отношений, эти условия должны содержать логические операции, которые и объединяют отношения. В С++ имеются следующие логические операции:

a	b	!a	a && b	a || b
true	true	false	true	true
true	false	false	false	true
false	true	true	false	true
false	false	true	false	false

Пример использования в инструкции if-else:

```
if ((score>0) && (score<10))
cout<<"счет в пределах от 0 до 10.\n"; else
cout<<"счет выходит за рамки от 0 до 10.\n";
```

Если значение переменной score больше 0 и меньше 10, то выполняется первая cout-инструкция, в противном случае выполняется вторая cout-инструкция.

Пример использования в инструкции if-else.

```
if ((x==l) || (x==y))
cout<<"x равно 1 или y\n"; else
cout<<"x не равно ни 1, ни y\n";
```

Если значение переменной x равно 1 или значению y, то выполняется первая cout-инструкция; в противном случае выполняется вторая cout- инструкция.

Если в логическом выражении круглые скобки опущены, компьютер группирует элементы в соответствии с **приоритетом операций.** Приоритеты некоторых операций в языке C++ приведены в таблице. Говорят, что оператор имеет **более высокий приоритет,** если по умолчанию он выполняется до другого оператора. Бинарные операторы с одинаковым приоритетом выполняются слева направо; унарные операторы с одинаковым приоритетом — справа налево.

Унарные операторы	+, -, ++, --, !	Строки таблицы расположены в порядке понижения приоритета
Бинарные арифметические операторы	*, /, %	
Бинарные арифметические операторы	+, -	
Логические операторы: <, >, <=, >=	<, >, <=, >=	
Логические операторы: ==, ! =	==, !=	
Логический оператор	& &	
Логический оператор	||	

Иногда в языке C++ в качестве логических значений используются целые числа. В частности, C++ преобразует целое значение 1 в true, а 0 — в false. На самом деле ситуация несколько сложнее, нежели просто использование 1 и 0 в качестве значений true и false соответственно. На самом деле компилятор трактует всякое отличное от нуля значение как true, а 0 — как false. Такое преобразование не создает каких-либо проблем, если не допускать ошибок при составлении логических выражений. До тех пор пока вы не допускаете ошибок в логических выражениях, вам не требуется знать, как логические и целые значения преобразуются друг в друга. Однако эта информация может понадобиться при отладке программ. Из-за этой ситуации компилятор не всегда сообщает об ошибке. Рассмотрим примеры таких ошибок.

Многократные неравенства

Не используйте в программе строк с многократными неравенствами, подобными следующим:

```
if (x<z<y) //Так делать  нельзя!.
cout<<"z находится в интервале между x и  y.";
```

При использовании приведенного фрагмента программа, скорее всего, откомпилируется и запустится, но результат ее работы, несомненно, будет

неправильным. Проблема не ограничивается только одним знаком < и возникает при использовании любого оператора сравнения. Правильная проверка на принадлежность переменной диапазону значений использует оператор &&:

```
if ((x<z) && (z<y))   // Правильная проверка
cout<<"z находится в интервале от x до y.";
```

Использование = вместо ==

К сожалению, в C++ можно написать множество инструкций, которые, с одной стороны, будут скомпилированы без ошибок, а с другой — будут выполнять не те действия, которые от них ожидают. Иногда можно просто не заметить неверную, запись, и это может привести к серьезным проблемам. На поиск ошибки программист может потратить немало времени, пока не поймет, что именно в программе сделано не так. Одной из распространенных ошибок является использование символа = вместо ==. Рассмотрим, например, инструкцию if-else, которая начинается следующим образом:

```
if   (x =  12)
Да_инструкция; else
Нет_инструкция
```

Вероятно, программист хотел проверить, равно ли значение x числу 12, и на самом деле он должен был написать не =, а ==. Вам кажется, что компилятор должен сообщить об ошибке— ведь выражение x = 12 не является условием, которое может или выполняться, или нет. Это инструкция присвоения, поэтому компилятор должен сообщить об ошибке! К сожалению, на самом деле это не так. В C++ x = 12 является выражением, которое имеет значение, и в этом смысле оно не отличается от таких выражений, как x + 12 или 2 + 3. Инструкция присвоения принимает значение выражения, которое находится слева от знака равенства, так что значение инструкции x = 12 равно 12. При обсуждении совместимости типов отмечалось, что ненулевые величины типа int могут быть преобразованы в логическое значение true, а потому в приведенном выше примере всегда будет выполняться первая инструкция *(Да_инструкция).*

Такую ошибку очень трудно обнаружить, потому что она *не похожа на ошибку!*. Но если поместить в левую часть соотношения число 12, как показано в следующем примере, то в случае использования одного знака равенства вместо двух компилятор тут же обнаружит ошибку.

```
if(12==x)
Да_инструкция; else
Нет_инструкция;
```

Помните о том, что использование одного знака равенства = вместо двух == является широко распространенной ошибкой, причем компилятор в таких случаях не выдает сообщений об ошибке, а обнаружить ее самому очень трудно. В C++ многие выполняемые инструкции могут служить выражениями других видов, в частности логическими выражениями в инструкции if-else. Так, если инструкцию присвоения поместить там, где должно быть логическое выражение, то она будет интерпретирована именно так, как ожидается, т.е. как логическое выражение. Конечно, формально результат "проверки" такого условия будет абсолютно

корректным, но совершенно не тем, который ожидает допустивший опечатку программист.

При использовании оператора if следует помнить:

- Если на какой-нибудь ветке надо выполнить несколько действий, эти действия надо заключать в операторные скобки { };
- Если на ветке «Да» используются операторные скобки, перед служебным словом else нельзя ставить точку с запятой;
- Если условный оператор содержит другой условный оператор, то ветка else относится к ближайшему условию. Поэтому, если при вложении операторов if какой-то оператор имеет не полную форму, рекомендуется использовать операторные скобки.
- Если выражение, которое записывается в условии длинное, имеет смысл записать условие как отдельный оператор присваивания. В этом случае следует дополнительно объявить переменную булевского типа и записать туда это выражение.

Примеры

Пример 1.

Вычислить A:

$a = \frac{b+c}{2 \cdot c}$, где $c = 4 \cdot b - 20$

Для произвольного b.

Исходные данные: b – вещественного типа.

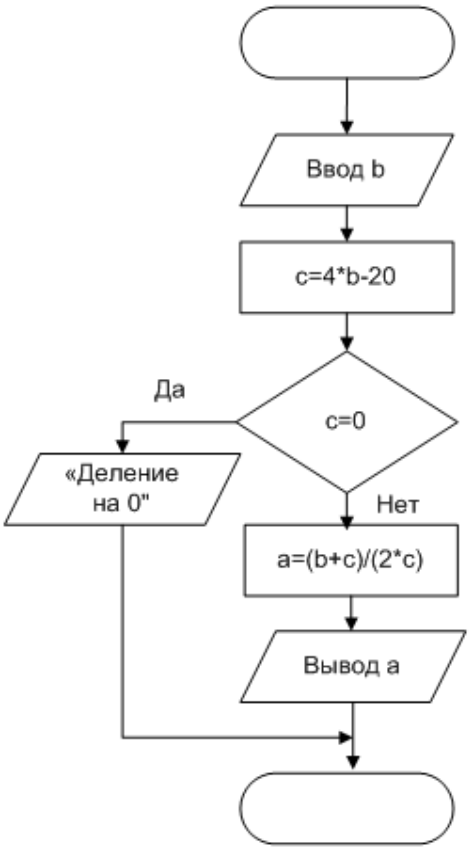
Результат: b – вещественного типа.

Сначала надо вычислить b, которая является частью выражения для вычисления a. При вычислении a может возникнуть ошибка – деление на ноль. Следовательно, перед делением надо выполнить проверку. Если знаменатель равен нулю - прекратить вычисление и выйти из программы.

Тестовый пример:

a. при b=1 a=0.468.

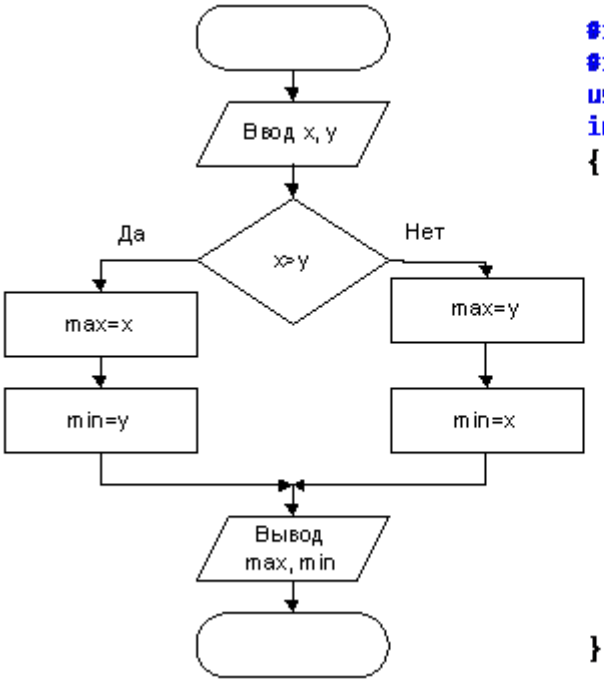
b. при b=5, выводится сообщение «Деление на ноль»




```
#include "stdafx.h"
#include <iostream>
#include <conio.h>
#include <clocale>
using namespace std;
int main()
{   double a,b,c;
     setlocale(LC_ALL, "Russian_Russia.1251");
    cout<<"b="; cin>>b;
        c=2*b-20;


if (c==0) cout<<"Деление на 0"<<endl;
        else


        {a=(b+c)/(2*c);


            cout<<"a="<<a<<endl;


         }
        _getch();
        return 0;
}
```

Пример 2.

Даны два произвольных числа. Определить максимальное и минимальное из этих чисел.




```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{   double x,y,max,min;
      cout<<"x="; cin>>x;
      cout<<"y="; cin>>y;
      if(x>y) {max=x;
            min=y;}
      else {max=y;
            min=x;}


      cout<<"max="<<max<<endl;
      cout<<"min="<<min<<endl;
      _getch();
      return 0;
}
```

Исходные данные: x и y – вещественного типа.

Результат: max и min – вещественного типа.

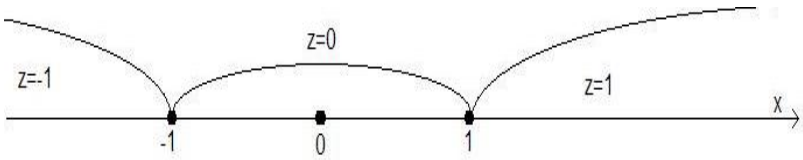
Тестовый пример:

при x=5, y=10, max=10, min=5.

Пример 3.

Вычислить z:

$$z = \begin{cases} -1, \text{ если } & x < -1 \\ 0, \text{ если } & -1 \ge x \ge 1 \\ 1, \text{ если } & x > 1 \end{cases}$$



Для произвольного x.

Исходные данные: x – вещественного типа.

Результат: z – вещественного типа.

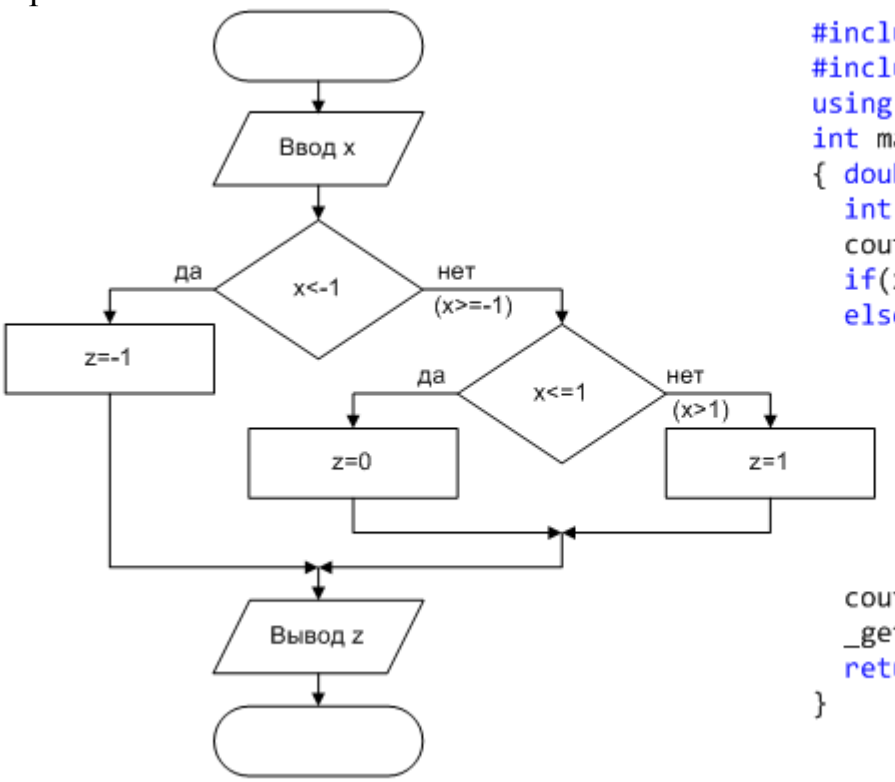
Необходимо определить в какую часть числовой оси попадает x

Тестовый пример:

a. при x=-10 z=-1;

b. при x=1 z=0;

c. при x=5 z=1.




```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{ double x;
  int z;
  cout<<"x="; cin>>x;
  if(x<-1) z=-1;
  else
     if(x<=1) z=0;
     else z=1;
  cout<<"z="<<z<<endl;
  _getch();
  return 0;
}
```

Пример 4.

Определить попадает ли точка с координатами (a,b) в заштрихованную область представленную на рисунке.

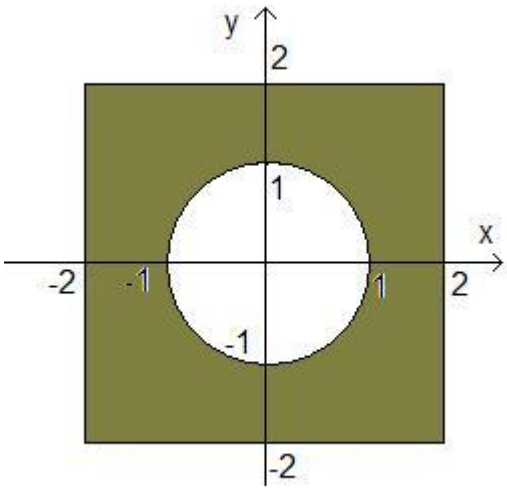
Исходные данные: a и b координаты точки– вещественного типа, радиус окружности – 1, половина стороны квадрата – 2.

Результатом будет сообщение попадает ли точка в данную область.

Тестовый пример:

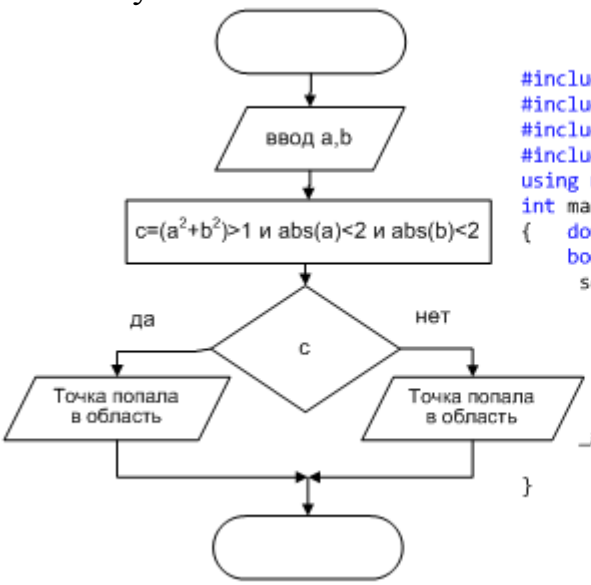
a. при x=0.5, y=0.5, точка не попала в область;

b. при x=1.2, y=-1.5, точка попала в область;



c. при x=3, y=-4, точка не попала в область.

Формула окружности $x^2+y^2=r^2$, где r – радиус окружности. Для точек вне окружности выполняется неравенство: $x^2+y^2>r^2$, если центр окружности имеет координаты (0,0) В квадрат попадают точки, для которых выполняется условие |x|<h/2 и |y|<h/2, где h – сторона квадрата и центр квадрата имеет координаты (0,0). Все эти условия должны выполняться одновременно, чтобы точка попала в заштрихованную область.



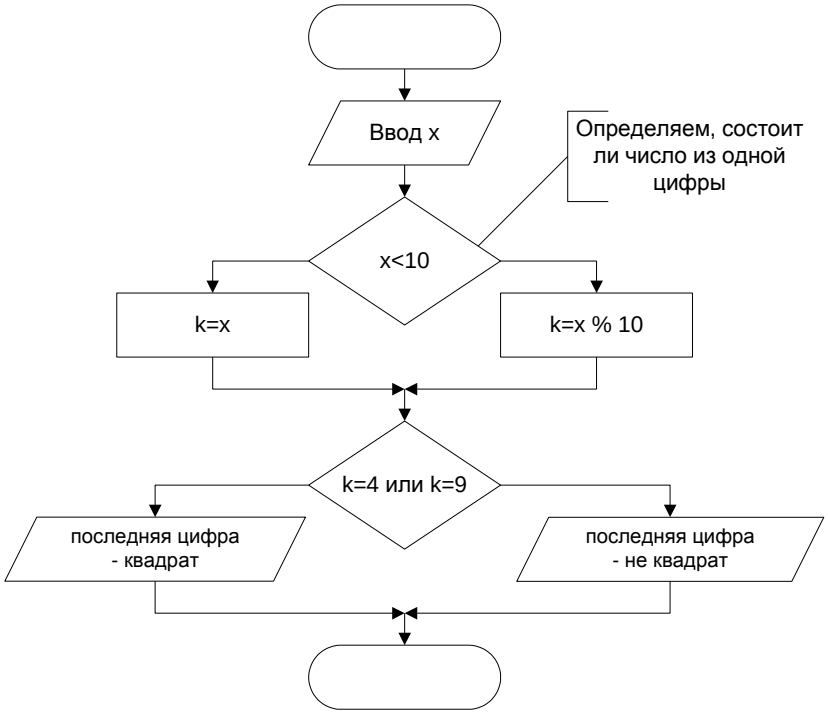

```
#include <stdio.h>
#include <iostream>
#include <conio.h>
#include <clocale>
using namespace std;
int main()
{   double a,b;
    bool c;
     setlocale(LC_ALL, "Russian_Russia.1251");
         cout<<"a="; cin>>a;
         cout<<"b="; cin>>b;
         c=(a*a+b*b)>1 && fabs(a)<2 && fabs(b)<2;
         if (c) cout<<"Точка попала в область"<<endl;
            else cout<<"Точка не попала в область"<<endl;
_getch();
   return 0;
}
```

Пример 5.

Дано целое число. Определить, является ли последняя цифра данного числа квадратом другого числа.

Исходные данные: Заданное число x – целого типа.

Результатом будет сообщение, является ли последняя цифра квадратом другого числа.



Среди чисел меньших 10 имеется только два числа, которые являются квадратами другого числа – это 4 и 9. Если заданное число больше 9, то надо выделить последнюю цифру. Обозначим эту цифру k. k можно получить как остаток от деления данного числа на 10. Если заданное число меньше 10, то k и есть данное число.

Тестовый пример:

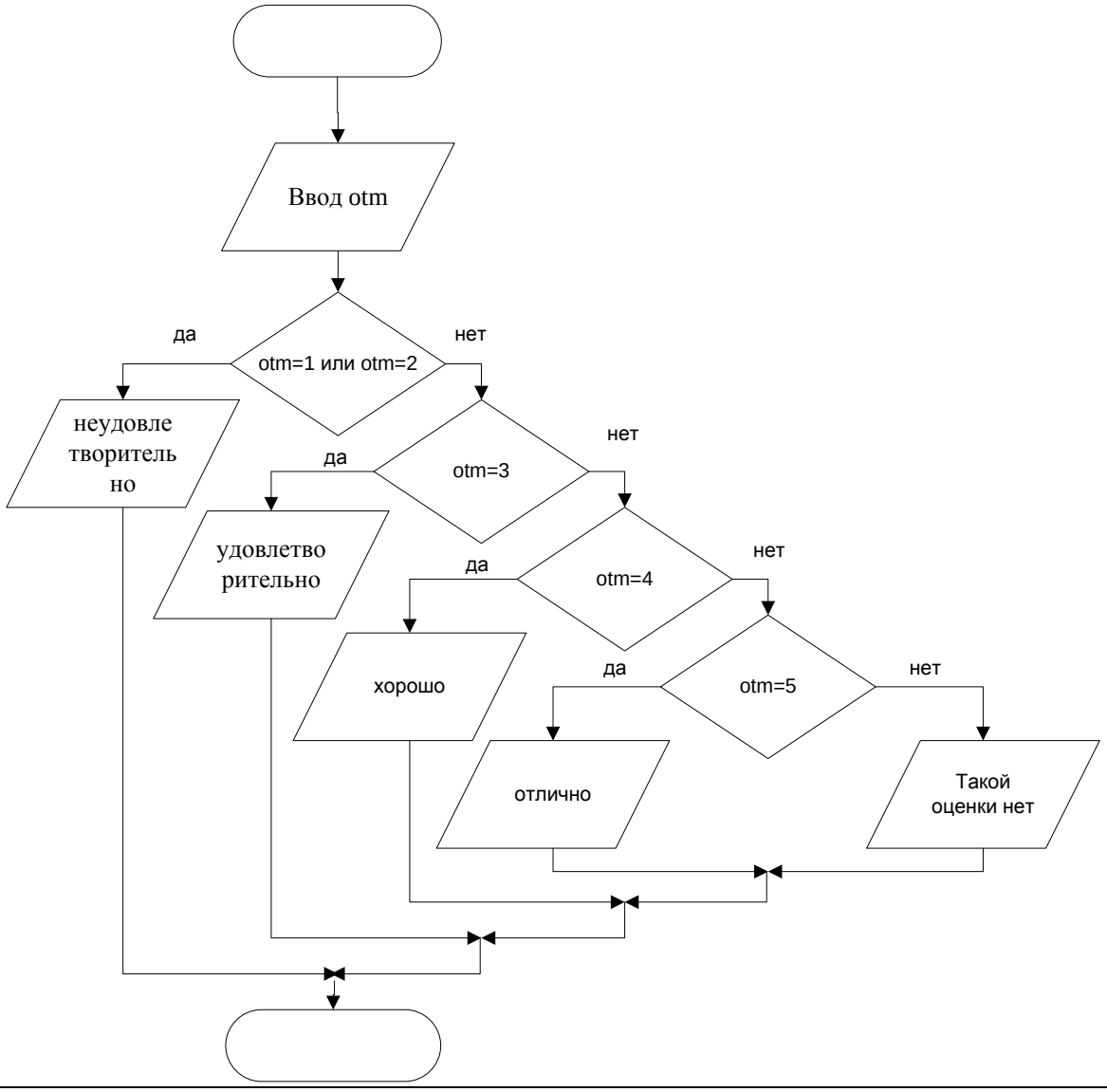
a. при x=8, последняя цифра не квадрат;
b. при x=9, последняя цифра квадрат;
c. при x=41, последняя цифра не квадрат;
d. при x=129, последняя цифра квадрат.

Пример 6.

Ввести оценку в виде числа и вывести в виде текста.

Исходные данные: Оценка otm – целое число.

Результат: оценка в виде сообщения: *неудовлетворительно*, *удовлетворительно*, и т.д. Если введенная оценка не попадает в интервал от 1 до 5, вывести сообщение, что такой оценки нет.



Задание 1 Ввести и отладить программу для примера 5

.

Контрольные вопросы

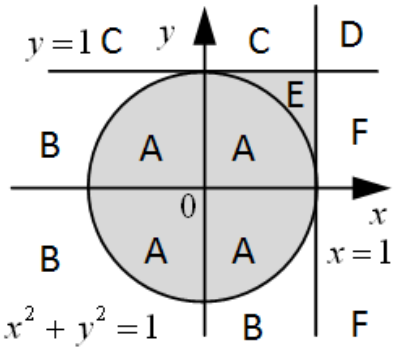
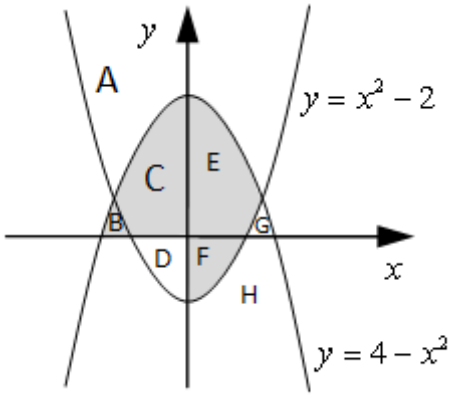
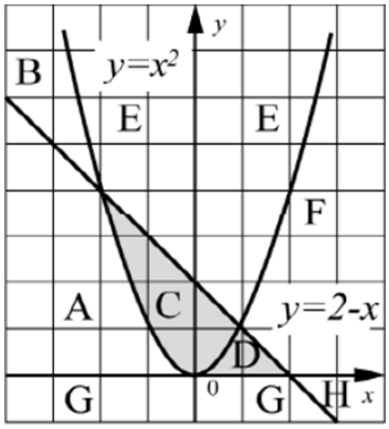
1. В какой ситуации в условном операторе используются {}.
2. Почему при записи (x<z<y) не будет сообщения об ошибке.

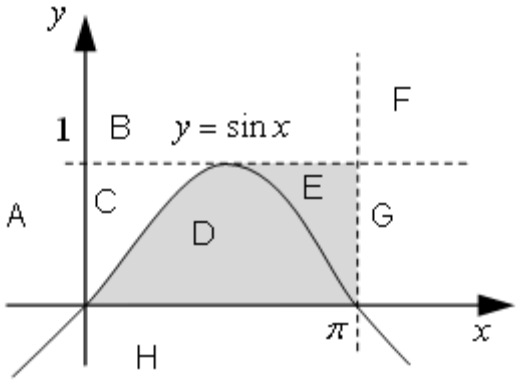
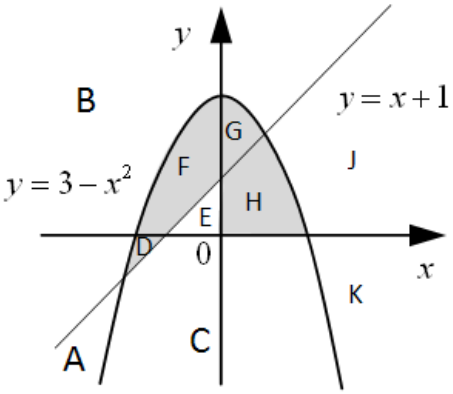
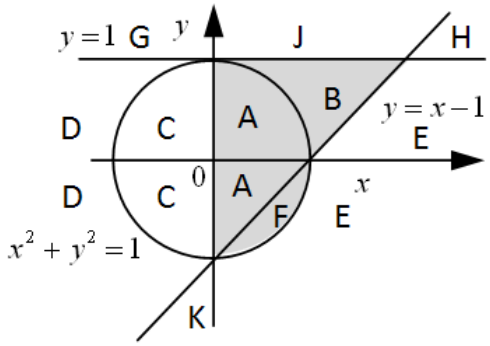
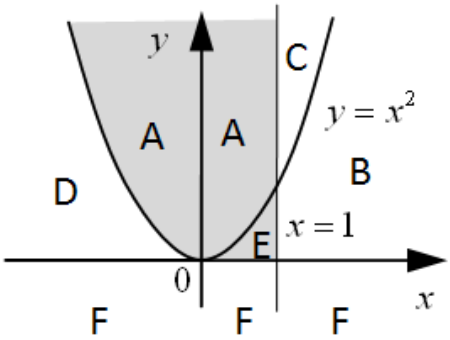
3. Расставьте в порядке понижения приоритета следующие операции: +, &&, >=, ||, *, <, %.
4. Почему в примере 3 нет проверки x>1.
5. Изобразите блок-схему примера 5 без использования сложного условия.
6. По какой ветке будет выполняться алгоритм примера 2, если x и y равны.

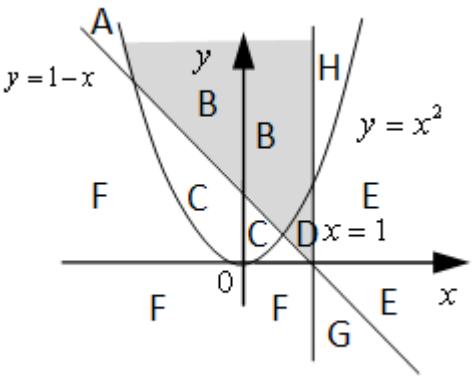
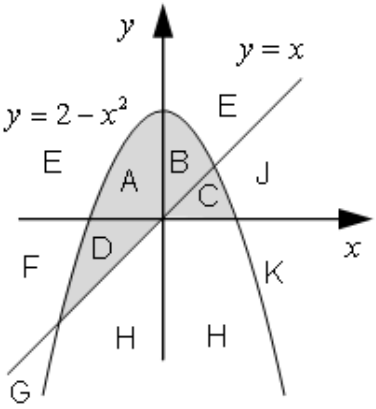
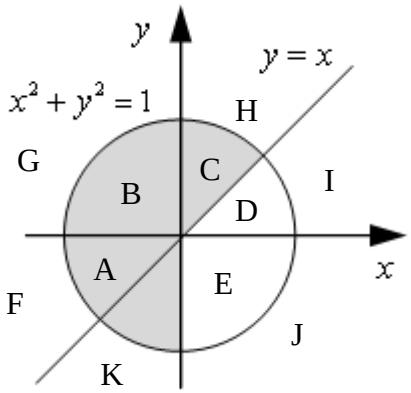
Индивидуальные задания

Требуется написать программу, которая вводит с клавиатуры координаты точки на плоскости (x, y – вещественные числа) и определяет принадлежность точки заштрихованной области, включая ее границы.

При составлении таблицы контрольных примеров отдельно пропишите проверку попадания точки в области, обозначенные на рисунке заглавными латинскими буквами.

№ вар.	Описание задания
1	
2	
3	

4	 y 1 B y = sin x F A C E G D π x H
5	 y B y = x + 1 G J y = 3 − x² F H E D 0 x K A C
6	 y = 1 G y J H B y = x − 1 D C A E 0 x D C A F E x² + y² = 1 K
7	 y C y = x² A A B D x = 1 E 0 x F F F

8	 A y H y = 1 − x B B y = x² F C E C D x = 1 0 x F F E G
9	 y y = x y = 2 − x² E E B A J C x D F K H H G
10	 y y = x x² + y² = 1 H G C I B D x A E F J K